Lezione 3 - Hooks

Introduzione

Abbiamo visto nella scorsa lezione cosa sono le props e come usarle.
Ora vediamo gli hooks e come usarli.
Prima pero di capire cosa sono gli **hooks** e perché sono così importanti, partiamo da una semplice osservazione:

- cosa succede in React quando cambia il valore di una variabile?

Immaginiamo di creare un componente che dichiara una variabile con valore iniziale `'ciao'` e una funzione che ne modifica il contenuto in `'arrivederci'`.
All'interno del `return`, inseriamo un bottone che, al click, richiama quella funzione:

```
const Messaggio = () => {
    let testo = "ciao";

    function onHandleClick() {
        testo = "arrivederci";
        console.log(testo);
    }

    return (
        <div>
              <p>{testo}</p>
              <button onClick={onHandleClick}>Cambia testo</button>
        </div>
    );
}
```

Quando proviamo ad eseguire questo codice e apriamo la console, noteremo che il valore della variabile **effettivamente cambia in memoria** (nella console vedremo `"arrivederci"`).
Tuttavia, **il testo visualizzato sullo schermo non si aggiorna**.
Questo accade perché React non "sa" che deve rifare il rendering del componente: il cambiamento della variabile non innesca automaticamente un aggiornamento della UI.

Ed è proprio qui che entrano in gioco gli **Hooks**.

Cosa sono gli Hooks

Gli *Hooks* sono funzioni speciali introdotte da React per permetterci di:

- ✅ **Gestire lo stato interno dei componenti (ovvero, memorizzare e aggiornare dati nel tempo);**
- ✅ **Gestire effetti collaterali (side effects) che avvengono dopo il rendering, come chiamate API o modifiche al DOM;**
- ✅ **Fare tutto questo direttamente all'interno dei componenti funzionali, senza dover ricorrere alle classi.**

In altre parole, gli Hooks ci consentono di:

- **rendere "reattivi" i componenti funzionali, ovvero capaci di reagire ai cambiamenti di stato aggiornando automaticamente la UI.**

✓ **I vantaggi dell'uso degli hooks sono :**

- Possiamo **decidere cosa mostrare all'utente** in base ai dati attuali;
- Descriviamo **come deve apparire l'interfaccia** in base allo stato, e React si occupa di renderla coerente;
- Rendiamo i componenti **più semplici, leggibili e modulari** rispetto ai vecchi componenti a classe.

I principali Hooks integrati (built-in)

React mette a disposizione diversi Hooks integrati.
Tra i più comuni troviamo:

1. `useState()` → **per gestire lo stato interno dei componenti.**
2. `useEffect()` → **per gestire gli effetti collaterali (ad es. fetch API, timers, ecc.).**
3. `useContext()` → **per accedere ai dati condivisi nel contesto dell'applicazione.**
4. `useReducer()` → **per gestire stati complessi con logiche di aggiornamento più articolate.**
5. `useRef()` → **per accedere direttamente a elementi del DOM o mantenere valori persistenti senza triggerare un rendering.**

Quello che impareremo nei prossimi passaggi sarà :

- Costruire un componente “stateful” (cioè con stato interno)
- Usare l’Hook `useState()` per creare e aggiornare uno stato
- Inizializzare e modificare i valori di stato
- Definire **event handler** per rispondere alle azioni dell’utente
- Utilizzare **funzioni di callback** per aggiornare correttamente lo stato
- Gestire lo stato anche con **array** e **oggetti**

Modificare un componente con lo State hook (`useState()`)

Quindi come detto poco sopra, quando creiamo un componente React, non abbiamo a disposizione costruttori o altri metodi (come accade nei componenti a classe) per **inizializzare e gestire lo stato interno**.
Nei componenti funzionali, questa necessità viene soddisfatta tramite una tecnica specifica: **gli Hooks**.

In particolare, l’Hook che utilizziamo per gestire lo stato è `useState()` — il più comune e fondamentale tra tutti.

Lo `useState()`

`useState()` è una funzione fornita da React che ci permette di:

- **Memorizzare un valore nello stato interno di un componente;**
- **Aggiornare quel valore e fare in modo che il componente venga nuovamente renderizzato ogni volta che lo stato cambia.**

In pratica, quando vogliamo che la nostra interfaccia utente *reagisca ai cambiamenti dei dati*, `useState()` è lo strumento che ce lo consente.

Importazione dell'Hook

Essendo un **named export** della libreria React, deve essere importato tramite **destrutturazione**, in questo modo:

```
import React, { useState } from "react";
```

Come funziona

Quando richiamiamo `useState()`, la funzione restituisce **un array con due valori**:

1. **Lo stato corrente** → **rappresenta il valore attuale dello stato.**
2. **La funzione setter dello stato** → **è la funzione che consente di aggiornare tale valore.**

Possiamo estrarre questi due valori utilizzando la **destrutturazione degli array**, come segue:

```
const [currentState, setCurrentState] = useState();
```

Dove:

- `currentState` → **rappresenta il valore attuale dello stato.**
- `setCurrentState` → **è la funzione che utilizziamo per modificarlo.**

> **Nota:**
>
> Il parametro passato a `useState()` è il **valore iniziale dello stato**, che può essere di qualsiasi tipo: stringa, numero, booleano, oggetto o array.

Cosa succede internamente

Se proviamo a stampare in console il valore di `useState`, noteremo che si tratta di una **funzione**.
Quando le passiamo un valore iniziale (ad esempio una stringa o un numero), React ci restituisce **un array** contenente:

- **Il valore iniziale stesso;**
- **Una funzione che serve per aggiornare lo stato.**

Questo meccanismo consente a React di:

- ***Tenere traccia* dei valori di stato;**
- ***Aggiornare automaticamente* il rendering del componente quando questi cambiano.**

Esempio pratico 1: Toogle di stato

```
import React, { useState } from "react";

const Toggle = () => {
    // Inizializziamo lo stato con valore undefined (può contenere
qualsiasi tipo)
    const [toggle, setToggle] = useState();

    return (
        <div>
            <p>The toggle is {toggle}</p>

          {/* La funzione setter viene richiamata sugli eventi
onClick */}
            <button onClick={() => setToggle("On")}>On</button>
            <button onClick={() => setToggle("Off")}>Off</button>
        </div>
    );
}
```

Cosa succede:

- Quando l'utente clicca su un pulsante, la funzione `setToggle()` aggiorna il valore dello stato.
- React rileva il cambiamento e **ricostruisce il rendering** del componente, aggiornando il testo mostrato nella pagina.

Esempio pratico 2 – Cambiare il colore dello sfondo

Vediamo ora un secondo esempio più completo, che mostra come usare `useState()` per aggiornare dinamicamente lo stile di un componente.

```
import React, { useState } from "react";

export default const ColorPicker = () => {
    // 1️⃣ Inizializziamo lo stato con il colore 'Tomato'
    const [color, setColor] = useState("Tomato");

    // 2️⃣ Creiamo un oggetto di stile che utilizza il valore di
stato
    const divStyle = { backgroundColor: color };

    return (
```

```
        <div style={divStyle}>
            <p>The color is {color}</p>

            {/* 3️⃣ Al click di ogni bottone aggiorniamo lo stato con un nuovo colore */}
            <button onClick={() => setColor("Aquamarine")}>Aquamarine</button>
            <button onClick={() => setColor("BlueViolet")}>BlueViolet</button>
            <button onClick={() => setColor("Chartreuse")}>Chartreuse</button>
            <button onClick={() => setColor("CornflowerBlue")}>CornflowerBlue</button>
        </div>
    );
}
```

Analisi passo-passo:

1. `useState("Tomato")` imposta il valore iniziale dello stato.
2. La variabile `color` rappresenta il colore attuale;
3. `setColor()` aggiorna lo stato quando l'utente clicca un bottone;
4. React effettua automaticamente il **re-rendering del componente**, aggiornando il colore di sfondo in base al nuovo valore.

Alcune regole fondamentali sugli Hooks

Per utilizzare correttamente gli Hooks, è importante ricordare alcune regole base:

- Tutti gli Hooks **devono iniziare con la parola** `use` (es. `useState`, `useEffect`, ecc.);
- Possono essere richiamati **solo all'interno di componenti funzionali o di altri Hooks personalizzati**;
- Non devono essere chiamati **all'interno di cicli, condizioni o funzioni annidate**:
  - **vanno dichiarati sempre a livello superiore del componente (sopra il `return` statement);**

- **Il nome del componente React deve iniziare con lettera maiuscola, altrimenti React non lo riconosce come tale**.

Inizializzare lo stato

Abbiamo visto che **l'Hook** `useState()` **ci permette di creare e gestire lo stato all'interno di un componente React.**
Ma cosa succede se vogliamo controllare **il valore iniziale** dello stato, oppure utilizzare tipi di dati diversi da una semplice stringa?

Proprio come possiamo usare `useState()` per memorizzare una stringa, possiamo usarlo anche per qualsiasi **tipo di dato primitivo:**

- booleano
- numero (integer, float)
- stringa
- ecc.
  Oppure **per collezioni di dati:**
- array
- oggetti.

Esempio di stato booleano

Osserviamo il seguente componente:

```
import React, { useState } from 'react';

const ToggleLoading = () => {
    const [isLoading, setIsLoading] = useState();

    return (
        <div>
            <p>The data is {isLoading ? 'Loading' : 'Not Loading'}
</p>
            <button onClick={() => setIsLoading(true)}>Turn Loading
On</button>
            <button onClick={() => setIsLoading(false)}>Turn Loading
Off</button>
        </div>
    );
}
```

In questo esempio:

- il componente `ToggleLoading` utilizza come stato un **valore booleano**,
  - rappresentato dalla variabile `isLoading`.

> **✏ Nota: l'uso degli `useState()` booleani**
>
> I booleani sono spesso usati in React per indicare se un dato è stato caricato oppure no, se un elemento è visibile o nascosto, o se un'azione è attiva o disattivata.

- Nel codice, **la funzione `setIsLoading()` riceve di volta in volta i valori `true` o `false`, che determinano cosa mostrare sullo schermo.**

Inizializzare lo stato con un valore predefinito

Questo codice funziona già correttamente, ma potremmo voler specificare che, **al primo rendering**, lo stato `isLoading` parta già impostato su `true`.

Per farlo, basta passare un valore iniziale come **argomento della chiamata a `useState()`**:

```
const [isLoading, setIsLoading] = useState(true);
```

In questo modo, React saprà che il valore iniziale dello stato deve essere `true`.
Da quel momento in poi, lo stato potrà essere modificato liberamente tramite la funzione `setIsLoading()`.

Come React gestisce il valore iniziale

Quando inizializziamo uno stato con `useState(valoreIniziale)`, React segue un comportamento preciso:

1. **Durante il primo render:**
   - **React usa il valore passato come argomento per impostare lo stato iniziale.**
2. **Quando chiamiamo la funzione setter:**
   - **React ignora l'argomento iniziale e utilizza invece il nuovo valore fornito.**

3. **Nei render successivi:**
   - **React mantiene il valore aggiornato dell'ultimo render precedente.**

E se non inizializziamo lo stato?

Se non passiamo alcun valore a `useState()`, durante il primo render **il valore corrente dello stato sarà** `undefined`.
Questo non causa errori, ma può rendere il codice meno leggibile o ambiguo per chi lo consulta.

Per questo motivo, è buona norma **inizializzare sempre lo stato in modo esplicito**, anche solo con `null` o con un valore "vuoto" del tipo previsto.
Esempio:

```
// Stato inizializzato a null
const [user, setUser] = useState(null);
```

Buone pratiche di inizializzazione

La scelta del valore iniziale dipende dal tipo di dato che vogliamo gestire e dal comportamento del componente.
Possiamo seguire alcune regole generali:

1. Caso 1 — Tipo di dato non definito a priori

Se non sappiamo che tipo di dato lo stato conterrà in futuro, è consigliabile inizializzare con `undefined`**.**
Questo approccio esplicita che il valore non è ancora stato impostato e offre massima flessibilità.

Motivazioni:

- **Chiarezza semantica:** `undefined` **comunica chiaramente che il valore non è stato ancora assegnato.**
- **Flessibilità tipologica: lo stato potrà contenere in seguito qualsiasi tipo di dato (stringa, numero, array, oggetto...).**
- **Debug più chiaro: in console o negli strumenti di sviluppo,** `undefined` **segnala in modo esplicito che lo stato è ancora "vuoto".**

```
const [data, setData] = useState();
```

2. Caso 2 — Tipo di dato conosciuto

Se invece sappiamo già che tipo di dato lo stato conterrà, è meglio **inizializzarlo coerentemente** con il suo tipo naturale:

```
const [items, setItems] = useState([]);    // sempre un array
const [user, setUser] = useState({});      // sempre un oggetto
const [count, setCount] = useState(0);     // sempre un numero
const [isActive, setIsActive] = useState(false); // sempre un
booleano
```

Questo approccio:

- **Evita controlli aggiuntivi nel codice (`if (items) ...`);**
- **Migliora la leggibilità;**
- **Riduce errori legati a tipi di dato inattesi.**

In Conclusione:

inizializzare correttamente lo stato è una buona pratica fondamentale per costruire componenti React più prevedibili, leggibili e facili da mantenere.

Ricorda sempre che lo stato non serve solo a *memorizzare dati*, ma a *guidare il comportamento* e *l'aspetto del componente* in base a quei dati.

Utilizzare il setter dello stato al di fuori del JSX

Uno dei casi più comuni in cui si utilizza lo stato (`useState`) è la gestione dei dati inseriti dall'utente nei campi di input.
Vediamo un esempio pratico in cui vogliamo memorizzare l'indirizzo email digitato in un campo di testo:

```
import React, { useState } from 'react';
```

```
export default const EmailTextInput = () => {
    const [email, setEmail] = useState('');

    const handleChange = (event) => {
        const updatedEmail = event.target.value;
        setEmail(updatedEmail);
  }

    return (
        <input value={email} onChange={handleChange} />
  );
}
```

Analisi passo per passo

1. **Inizializzazione dello stato**
   - Usiamo la *destrutturazione dell'array* restituito da `useState()` per ottenere due elementi:
     - `email` : **il valore corrente dello stato;**
     - `setEmail` : **la funzione setter che aggiorna quello stato.**
2. **Gestione del cambiamento**
   - Il campo `<input>` in JSX possiede un _event listener_ chiamato `onChange` .
     - **Questo evento si attiva ogni volta che l'utente digita qualcosa nel campo.**
   - **Quando ciò accade, React richiama automaticamente la funzione `handleChange` , passando come argomento un *oggetto evento* (`event`).**
3. **Aggiornamento dello stato**
   - **All'interno di `handleChange` , leggiamo il valore digitato (`event.target.value`) e lo memorizziamo nella variabile locale `updatedEmail` .**
   - **Infine, chiamiamo `setEmail(updatedEmail)` per aggiornare lo stato React, che a sua volta causa un *re-render* del componente, aggiornando il valore visualizzato nel campo di input.**

Perché definire l'event handler fuori dal JSX?

In precedenza abbiamo visto che è possibile scrivere event handler *inline*, ad esempio:

```
<input value={email} onChange={(e) => setEmail(e.target.value)} />
```

Questo approccio funziona perfettamente, ma **quando la logica dell'evento diventa più complessa (ad esempio include validazioni, condizioni, log o altre operazioni), è buona pratica separarla dal markup JSX.**

Separare la logica dalla struttura visiva rende il codice:

- **più leggibile e organizzato,**
- **più facile da testare,**
- **e più semplice da mantenere nel tempo.**

Questa tecnica segue il principio della **"*separation of concerns:*"**
ogni parte del codice ha una responsabilità chiara e limitata.

Scritture equivalenti dell'handler

Nel nostro esempio, la funzione `handleChange` può essere scritta in modi diversi ma equivalenti:

```
const handleChange = (event) => {
  const newEmail = event.target.value;
  setEmail(newEmail);
};
```

Oppure in forma più compatta:

```
const handleChange = (event) => setEmail(event.target.value);
```

E infine usando la **destrutturizzazione dell'oggetto evento**

```
const handleChange = ({ target }) => setEmail(target.value);
```

Tutte queste versioni si comportano allo stesso modo:
bisogna solo scegliere quella che si trova più leggibile.

> **Di norma, la terza è la più concisa e diffusa nei progetti React moderni.**

Esempio pratico: convalidare un numero di telefono

Vediamo ora un esempio più avanzato, dove gestiamo un input numerico controllando che l'utente inserisca solo cifre:

```
const validPhoneNumber = /^\d{1,10}$/;

const PhoneNumber = () => {
    const [phone, setPhone] = useState("");

    const handleChange = ({ target }) => {
        // target rappresenta l'elemento HTML <input> che ha
generato l'evento
        const newPhone = target.value; // valore digitato
dall'utente

        // Controlla che il valore sia composto solo da numeri (1-10
cifre)
        const isValid = validPhoneNumber.test(newPhone);

        // Se valido, aggiorna lo stato (React ri-renderizzerà il
componente)
        if (isValid){ setPhone(newPhone)};
  };

  return (
    <div className="phone">
      <label htmlFor="phone-input">Phone: </label>
      <input
        id="phone-input"
        value={phone}
        onChange={handleChange}
      />
    </div>
  );
}
```

Cosa succede nel dettaglio

1. **L'utente digita nel campo input.**
   - L'evento `onChange` si attiva e React chiama `handleChange()`.
2. **La funzione riceve un oggetto evento.**
   - **Tramite destrutturizzazione, accediamo a `target`, ovvero l'elemento HTML `<input>` che ha generato l'evento**.
3. **Lettura e validazione del valore.**
   - `target.value` **contiene il testo digitato**.
   - **Usiamo l'espressione regolare `validPhoneNumber` per verificare che siano solo cifre (massimo 10).**
4. **Aggiornamento dello stato condizionale.**
   - **Se il valore è valido (`isValid =** true `), viene chiamato` setPhone(newPhone)`.==
   - **In caso contrario, lo stato non viene modificato, e React ignora l'input errato**.
5. **Re-render automatico.**
   - Quando lo stato cambia, React aggiorna il componente e il campo mostra il nuovo valore.

In pratica, il flusso è:

> 🥷 Utente digita → 🧠 `onChange` si attiva → ⚙️ `handleChange` valida il valore → ✅ `setPhone()` aggiorna lo stato → 🔁 React ri-renderizza il componente.

📋 Comprendere `target.value` negli event handler di React

Quando utilizziamo eventi in React (come `onChange`, `onClick`, `onSubmit`, ecc.), **il framework richiama automaticamente una funzione di gestione dell'evento (detta *event handler*) passando come argomento un oggetto evento**.

Questo oggetto rappresenta **tutte le informazioni sull'evento che si è verificato**:
quale elemento l'ha generato, in quale momento, con quale valore, e così via.

Esempio generico:

```
function handleChange(event) {
  console.log(event); // Oggetto evento
}
```

Se questo handler è collegato a un campo `<input>`,
l'oggetto `event` contiene, tra le altre proprietà, un riferimento al campo stesso — ed è qui che entra in gioco `event.target`.

Cosa rappresenta `target`

La proprietà `target` dell'oggetto evento è un riferimento all'elemento DOM che ha generato l'evento.

In altre parole, `event.target`:

- **è il nodo HTML su cui l'utente ha interagito.**

Nel caso di un input:

```
<input onChange={handleChange} />
```

Cosa rappresenta `target.value`

La proprietà `.value` di un elemento `<input>` contiene il valore corrente del campo di testo.
Ogni volta che l'utente digita, cancella o modifica qualcosa, il valore cambia, e `event.target.value` viene aggiornato di conseguenza.

Esempio pratico:

```
const handleChange = (event) => {
  console.log(event.target.value);
};
```

- Se l'utente digita `3`, la console mostrerà `"3"`.
- Poi digita `39` → la console mostrerà `"39"`.
- Poi `390` → `"390"`, e così via.

Questa è la modalità con cui React “ascolta” ciò che l’utente sta scrivendo, passo dopo passo.

ⓘ Perché salviamo `event.target.value` in una variabile locale

Spesso si vede codice di questo tipo in React:

```
const handleChange = ({ target }) => {
    const newPhone = target.value;
    setPhone(newPhone);
};
```

Qui accade quanto segue:

1. **Destrutturizzazione dell’oggetto evento:**
   Scrivendo `({ target })` invece di `(event)`, diciamo a JavaScript di estrarre direttamente la proprietà `target` dall’oggetto evento.

   È una forma abbreviata di:

   ```
   const handleChange = (event) => {
       const target = event.target;
           ...
       };
   ```

2. **Lettura del valore dell’input:**
   `target.value` rappresenta ciò che l’utente ha appena digitato nel campo `<input>`.
3. **Memorizzazione temporanea:**
   Assegniamo questo valore a una variabile locale, ad esempio `newPhone`, per poterlo usare facilmente in più punti della funzione (ad esempio per validarlo con una regex prima di aggiornare lo stato).
4. **Aggiornamento dello stato:**
   Se il valore è valido, chiamiamo `setPhone(newPhone)`, che aggiorna lo stato del componente.

Riassumendo:

- `event` → **l'oggetto evento React.**
- `event.target` → **il campo input che ha generato l'evento**.
- `event.target.value` → **il contenuto (testo) attualmente presente nel campo input.**

Puoi pensarla così:

Ogni volta che scrivi qualcosa in un input, React crea un "messaggio" (l'oggetto evento) che contiene il riferimento al campo (`target`) e il suo contenuto aggiornato (`target.value`).
L'event handler riceve questo messaggio e decide come reagire — ad esempio aggiornando lo stato con `setEmail(event.target.value)` o `setPhone(event.target.value)`.

Quindi il flusso dell'evento, dell'esempio fatto sopra può essere riassunto cosi:

1. **L'utente digita →** `onChange` **viene attivato.**
2. **React passa un oggetto evento alla funzione** `handleChange`**.**
3. `target` **rappresenta l'input HTML che ha generato l'evento.**
4. `target.value` **contiene ciò che l'utente ha digitato.**
5. **Se il valore è valido, lo stato** `phone` **viene aggiornato.**
6. **React effettua un nuovo render, mostrando il valore aggiornato nel campo.**

Nel caso di componenti controllati (come questo), il valore dell'input (`value={phone}`) non vive più nel DOM, ma nello stato React.
Ogni volta che digiti, React aggiorna lo stato e poi "restituisce" quel valore all'input, garantendo che l'interfaccia sia sempre sincronizzata con i dati interni del componente.

Aggiornare lo stato precedente in React

Nei paragrafi precedenti abbiamo visto come aggiornare lo stato di un componente
passando direttamente un nuovo valore, ad esempio:

```
setState(newStateValue);
```

Tuttavia, è importante sapere che **gli aggiornamenti di stato in React sono asincroni:**

- **ciò significa che, quando invochiamo una funzione setter (come `setCount()` o `setEmail()`), React *non aggiorna immediatamente* lo stato.**
  In alcuni casi, altre parti del codice possono quindi essere eseguite **prima** che il nuovo stato venga effettivamente applicato.

Perché questo comportamento?

Questo meccanismo non è un difetto:
è una scelta progettuale di React per ottimizzare le prestazioni del rendering.
Infatti, React può raggruppare più aggiornamenti di stato in un unico ciclo di rendering,
riducendo i costi computazionali.

Tuttavia, questo approccio può portare a un rischio:
⚠️ **l'utilizzo di valori di stato obsoleti, se il nuovo stato dipende dal valore precedente.**

La soluzione: usare una *callback function*

Quando il nuovo valore di stato dipende da quello precedente,
è **buona prassi** utilizzare la versione "funzionale" della funzione setter.

In altre parole, invece di passare un valore diretto,
passiamo a `setState()` una **funzione di callback** che riceve:

- **il valore precedente dello stato come argomento, e restituisce il nuovo valore.**

Esempio pratico

```
import React, { useState } from "react";

export default function Counter() {
    // Dichiarazione dello stato 'count' inizializzato a 0
    const [count, setCount] = useState(0);

    // Definizione dell'event handler che aggiorna lo stato
    const increment = () => setCount(prevCount => prevCount + 1);

    // Rendering del componente
    return (
        <div>
          <p>Wow, hai cliccato quel bottone {count} volte!</p>
          <button onClick={increment}>Clicca qui!</button>
        </div>
    );
}
```

Come funziona

Quando l'utente clicca sul bottone:

1. **React invoca la funzione `increment()`.**
2. A sua volta `increment()` chiama `setCount()` e le passa una callback.
3. **React, internamente, fornisce alla callback l'ultimo valore valido di `count`.**
4. **La callback restituisce il nuovo valore (`prevCount + 1`).**
5. **React aggiorna lo stato e ri-renderizza il componente con il valore aggiornato.**

ⓘ Perché non scrivere semplicemente `setCount(count +1)`

In molti casi funzionerebbe comunque, ma **non sempre in modo sicuro:** Se due o più aggiornamenti vengono eseguiti in rapida successione (ad esempio, due click molto ravvicinati o più update concatenati), potresti finire per usare un valore di `count` non ancora aggiornato.

Con la callback (`setCount(prevCount => prevCount + 1)`), invece, sei certo che React utilizzi l'ultimo valore effettivo di `count`, anche se più aggiornamenti sono in coda.

Esempio articolato : navigazione di un quiz

Vediamo ora un caso reale in cui è fondamentale usare lo stato precedente: nella navigazione di un quiz.

```
import React, { useState } from "react";

const QuizNavBar = ({ questions }) => {
  const [questionIndex, setQuestionIndex] = useState(0);

  // Event handler per tornare indietro
  const goBack = () =>
    setQuestionIndex(prevQuestionIndex => prevQuestionIndex - 1);

  // Event handler per andare avanti
  const goToNext = () =>
    setQuestionIndex(prevQuestionIndex => prevQuestionIndex + 1);

  // Booleani di controllo per disabilitare i pulsanti
  const onFirstQuestion = questionIndex === 0;
  const onLastQuestion = questionIndex === questions.length - 1;

  return (
    <nav>
      <span>Domanda #{questionIndex + 1}</span>
      <div>
        <button onClick={goBack} disabled={onFirstQuestion}>
          Indietro
        </button>
        <button onClick={goToNext} disabled={onLastQuestion}>
          Avanti
        </button>
      </div>
    </nav>
  );
};
```

- **Lo stato `questionIndex` tiene traccia della posizione corrente nel quiz.**
- Gli handler `goBack()` e `goToNext()` utilizzano **callback functions**: **in questo modo calcolano il nuovo valore (domanda precedente o**

successiva)
basandosi sul valore precedente.

- Le costanti `onFirstQuestion` e `onLastQuestion` sono booleani che servono per disabilitare i pulsanti ai limiti del quiz.

Quando lo stato diventa "stale" (obsoleto)

Fino a questo punto abbiamo visto come React aggiorna lo stato all'interno di callback asincrone utilizzando la forma funzionale del setter, cioè passando come argomento `prevNomeStato` per accedere al valore precedente.
Questa tecnica non è solo una questione di praticità: **è necessaria per evitare che lo stato diventi *stale* (cioè obsoleto)**.

Esempio Pratico:

```
import React, { useState, useEffect } from "react";

export default function StaleStateExample() {
  const [count, setCount] = useState(0);

  // Simuliamo un aggiornamento ritardato con setTimeout
  const incrementWithTimeout = () => {
    setTimeout(() => {
      // ❌ Qui 'count' potrebbe essere obsoleto se clicchiamo più
volte rapidamente
      setCount(count + 1);
      console.log("Stale count:", count);
    }, 2000);
  };

  // Soluzione corretta: usare la forma funzionale
  const incrementSafely = () => {
    setTimeout(() => {
      // ✅ Qui 'prevCount' è sempre aggiornato
      setCount(prevCount => prevCount + 1);
      console.log("Updated count:", count);
    }, 2000);
  };

  return (
    <div>
      <p>Contatore: {count}</p>
      <button onClick={incrementWithTimeout}>
```

```
        Incrementa (rischio stale)
      </button>
      <button onClick={incrementSafely}>
        Incrementa in sicurezza
      </button>
    </div>
  );
}
```

Spiegazione passo passo:

1. `incrementWithTimeout` usa `count + 1` direttamente.
   - Se clicchi più volte in rapida successione, alcuni aggiornamenti perderanno il conteggio perché `count` catturato all'interno del `setTimeout` è obsoleto (*stale*).
2. `incrementSafely` usa `setCount(prevCount => prevCount + 1)`.
   - La callback riceve sempre l'ultimo valore aggiornato (`prevCount`), indipendentemente da quanti clic vengano fatti prima che il `setTimeout` venga eseguito.
3. In pratica, la forma funzionale garantisce **coerenza dello stato anche in scenari asincroni** come `timeout`, `interval`, o listener esterni.

`useState`: valore vs functional return

Come abbiamo già detto in precedenza la funzione `useState()` restituisce 2 elementi:

1. Il valore attuale dello stato (`currentValue`)
2. Una funzione `set` che serve per aggiornare autonomamente quel valore (`setterState`)
   Ad esempio:

```
import React, { useState } from "react";

const Contatore = () => {
  const [contatore1, setContatore1] = useState(0);
  let contatore2 = 0;

  const aumenta1 = () => {
    setContatore1(valoreAttuale => {
      console.log(contatore1); // Mostra il valore precedente
      return valoreAttuale + 1;
```

```
    });
  };

  const aumenta2 = () => {
    contatore2++;
    console.log(contatore2); // Variabile normale, non legata al
rendering
  };

  const diminuisci2 = () => {
    contatore2--;
    console.log(contatore2);
  };
};
```

Spiegazione:

- **Nel primo caso usiamo `useState` → infatti React gestisce lo stato e il rendering**
- **Nel secondo caso usiamo una variabile locale (`contatore2`) → in questo caso React non gestisce lo stato e il rendering**

La differenza chiave è:

> `useState` restituisce **il valore dello stato** e una funzione `set` che aggiorna automaticamente il componente.
> Cambiando direttamente una variabile normale, React **non aggiorna il componente**.

Per ovviare a questo problema si usano due approcci differenti per aggiornare lo stato:

1. **Passando direttamente un nuovo valore**
2. **Passando una funzione di callback che riceve il valore precedente**

Esempio pratico

```
const Contatore = () => {
  const [contatore1, setContatore1] = useState(0);

  // Aggiornamento diretto
  const aumenta = () => {
    setContatore1(contatore1 + 1);
```

```
    console.log(contatore1);
  };

  // Aggiornamento con functional return
  const diminuisci = () => {
    setContatore1(valoreAttuale => {
      console.log(valoreAttuale);
      return valoreAttuale - 1;
    });
  };
};
```

- Da notare come entrambi funzionano in scenari semplici
- La differenza vera emerge quando lo stato viene aggiornato **asincronamente**

Che significa; assumiamo di avere un componente `Contatore` e di aggiungere a questo componente un `setTimeout` di 2 secondi per simulare un aggiornamento asincrono.

```
const Contatore = () => {
  const [contatore1, setContatore1] = useState(0);

  const aumenta = () => {
    setTimeout(() => {
      setContatore1(contatore1 + 1); // ❌ rischio stato "stale"
      console.log(`Contatore 1: ${contatore1}`);
    }, 2000);
  };
};
```

In questo caso se l'utente clicca più volte velocemente, alcuni click vengono ignorati.

Lo stato viene catturato al momento della definizione del time-out → diventa **stale**

La soluzione quindi è tramite il **functional return:**

```
const Contatore = () => {
  const [contatore1, setContatore1] = useState(0);

  const aumenta = () => {
    setTimeout(() => {
      // Metodo 1: return implicito (forma compatta)
```

```
      setContatore1(valoreAttuale => valoreAttuale + 1);

      // Metodo 2: return esplicito (forma estesa)
      // setContatore1(valoreAttuale => {
      //   return valoreAttuale + 1;
      // });

      console.log(contatore1);
    }, 2000);
  };
};
```

- Con la forma funzionale, React **usa sempre l'ultimo valore valido**
- Nessuno stato stale, anche con click ravvicinati o più aggiornamenti in coda

> **Nota didattica**
>
> React permette di usare sia la forma compatta che quella estesa negli aggiornamenti funzionali.
> Nell'esempio seguente è attivo solo il metodo compatto, mentre quello esteso è commentato per scopi illustrativi.

Implementare controlli tramite functional return

Inoltre il functional return è utile anche per implementare logiche aggiuntive.

Esempio pratico: limitare il valore massimo

```
const Contatore = () => {
  const [contatore1, setContatore1] = useState(0);

  const aumenta = () => {
    setContatore1(oldValue => {
      if (oldValue + 1 === 5) {
        return oldValue; // blocca il contatore a 4
      } else {
        return oldValue + 1;
      }
    });
  };
};
```

In questo caso:

- `oldValue` è sempre l'ultimo valore valido
- Possiamo applicare qualsiasi regola di controllo sullo stato.

🎟 Regola pratica:

Quando il nuovo valore di stato **dipende dal precedente**, **usa sempre il functional return** per evitare problemi di stato obsoleto (*stale*).

📋 Aggiornamenti funzionali: Return implicito Vs. Return esplicito

Quindi quando si aggiorna lo stato, abbiamo 2 modi per scrivere un aggiornamento funzionale:

1. **Forma compatta (return implicito):**

```
setState(prev => prev +1);
```

Questa è il return implicito; **una semplice arrow function:**
La funzione prende il valore precedente (`prev`) e restituisce direttamente il nuovo valore.
2. **Forma estesa (explicit return con blocco)**

```
setState(prev => {
  return prev + 1;
});
```

In questo caso:

- Si usano le parentesi graffe (`{}`) → la arrow function non restituisce automaticamente nulla.
- **Di conseguenza si deve usare lo statement `return` per restituire il nuovo valore.**

> **‹› Non c'è nessuna differenza nel risultato finale.**
> Entrambe le forme:
>
> - ricevono il **valore precedente** dello stato,
> - restituiscono un **nuovo valore**,

- evitano lo stato *stale*,
- garantiscono aggiornamenti coerenti.

L'unica differenza è sintattica, non funzionale.

ⓘ Quando scegliere una forma o l'altra?

1. Si usa la forma compatta(return implicito) quando si deve fare un aggiornamento semplice:

```
setState(prev => prev + 1);
setState(prev => prev.filter( ... ));
setState(prev => [ ... prev, nuovo]);
```

2. Si usa la forma estesa quando si ha bisogno di più logica:

```
setState(prev => {
if (prev === 10) {
   console.log("Limite raggiunto");
   return prev;
}
 return prev + 1;
});
```

La forma estesa permette condizioni, log. return multipli, ecc.

useState con Array

Nelle applicazioni realizzate con React capita spesso di dover gestire **liste di dati**, come elenchi di prodotti, utenti, messaggi, ingredienti, notifiche e così via. Per questo motivo gli array rappresentano uno dei modelli di dati più utilizzati quando si lavora con componenti che devono renderizzare insiemi di elementi all'interno del JSX.

Gli array in JavaScript si prestano particolarmente bene a questo scopo, perché consentono di scorrere, filtrare e trasformare gruppi di informazioni in modo semplice e intuitivo. In React, quando un componente deve visualizzare un

insieme dinamico di elementi oppure modificare una lista in risposta alle azioni dell'utente, l'uso degli array diventa naturale e immediato.

Prima di entrare nel dettaglio è importante ricordare un principio fondamentale di React:

- **Lo stato non deve mai essere modificato direttamente. Ogni aggiornamento deve avvenire creando un nuovo valore, ad esempio un nuovo array che sostituisce quello precedente.**

Esempio introduttivo

```
import React, { useState } from 'react';

// Array statico di condimenti disponibili per la pizza
const options = ['Bell Pepper', 'Sausage', 'Pepperoni',
'Pineapple'];

export default function PersonalPizza() {
    const [selected, setSelected] = useState([]);

    const toggleTopping = ({ target }) => {
        const clickedTopping = target.value;
        setSelected(prev => {
            // Controlla se il topping cliccato è già stato
selezionato
            if (prev.includes(clickedTopping)) {
                // Rimuove il topping dallo stato
                return prev.filter(t => t !== clickedTopping);
            } else {
                // Aggiunge il nuovo topping allo stato
                return [clickedTopping, ...prev];
            }
        });
    };

    return (
        <div>
            {options.map(option => (
                <button value={option} onClick={toggleTopping} key=
{option}>
                    {selected.includes(option) ? 'Remove ' : 'Add '}
{option}
```

```
                </button>
            ))}
            <p>Order a {selected.join(', ')} pizza</p>
        </div>
    );
}
```

In questo esempio utilizziamo due array con funzioni differenti:

1. `options` – dati statici
   - Contiene l'elenco dei condimenti disponibili.
   - L'array non subisce modifiche nel corso dell'esecuzione ed è quindi definito **al di fuori** della function component.
   - Questo evita di ricrearlo ad ogni render e migliora l'efficienza.
2. `selected` – dati dinamici
   - Rappresenta i condimenti scelti dall'utente.
   - Lo stato viene inizializzato come array vuoto tramite `useState([])` e si aggiorna ad ogni clic sui pulsanti.
   - Nel JSX il metodo `.map()` viene utilizzato per generare dinamicamente un pulsante per ciascun elemento contenuto in `options`.

Spiegazione passo-passo

- **Importazione delle dipendenze**
  `import React, { useState } from 'react';`
  - Importa React e l'hook `useState`, necessario per creare e gestire uno stato locale all'interno del componente.
- **Definizione dell'array statico**
  `const options = [...];`
  - Crea una lista di condimenti disponibili.
  - È definita **fuori** dal componente perché non cambia nel tempo e non deve essere ricreata ad ogni render.
- **Dichiarazione del componente**
  `export default function PersonalPizza() { ... }`
  - Definisce il componente funzionale che React renderizzerà.
- **Creazione dello stato locale**
  `const [selected, setSelected] = useState([]);`
  - Inizializza lo stato `selected` come array vuoto.
  - Conterrà i condimenti scelti dall'utente.

- `setSelected` è la funzione utilizzata per aggiornare tale stato.

- **Funzione di gestione del clic**
  `const toggleTopping = ({ target }) => { ... }`
  - Gestisce l'interazione con i pulsanti.
  - Estrae `target` dall'evento e ricava il valore del bottone cliccato.

  Logica interna:
  - `setSelected(prev => { ... })`
    Utilizza la forma con funzione per basarsi correttamente sul valore precedente dello stato.
  - `prev.includes(clickedTopping)`
    Verifica se l'elemento è già selezionato.
    - **Se sì** → lo rimuove con `.filter()`.
    - **Se no** → lo aggiunge creando un nuovo array `[clickedTopping, ...prev]`.

  Nota didattica:
  React richiede la creazione di un *nuovo* array ad ogni aggiornamento (immutabilità dello stato).

- **Render della lista di pulsanti**
  `{options.map(option => (...))}`
  - Itera sull'array statico `options` e genera un pulsante per ogni condimento.

  Caratteristiche dei pulsanti:
  - `value={option}` → passa il nome del condimento all'handler.
  - `onClick={toggleTopping}` → esegue la funzione di gestione.
  - `key={option}` → fornisce a React una chiave unica per la lista.
  - Il testo mostra *"Add"* o *"Remove"* in base alla presenza dell'elemento nello stato (`selected.includes(option)`).

- **Visualizzazione dell'ordine finale**
  `<p>Order a {selected.join(', ')} pizza</p>`
  - Concatena gli elementi dell'array `selected` in una stringa leggibile.
  - Se nessun topping è selezionato, la stringa risulta vuota.

📋 Flusso di esecuzione (runtime)

1. **Primo render**

- `selected` è `[]`. Vengono renderizzati 4 bottoni ("Add Bell Pepper", ecc.) e il paragrafo con `Order a pizza` (vuoto).

2. **Utente clicca su un bottone**

- Viene eseguito `toggleTopping(event)`. `clickedTopping` diventa, es. `"Pepperoni"`.
- `setSelected(prev => ...)` aggiorna lo stato restituendo il nuovo array. React programma un re-render.

3. **Re-render**

- `selected` ora contiene `"Pepperoni"`. I bottoni si aggiornano: il bottone per `"Pepperoni"` mostrerà `Remove Pepperoni`.
- Il paragrafo mostrerà `Order a Pepperoni pizza`.

4. **Se l'utente clicca di nuovo lo stesso bottone**

- L'handler rimuove `"Pepperoni"` con `filter`, lo stato torna a `[]` e la UI si aggiorna di conseguenza.

Aggiornare uno stato di tipo array

Come detto prima **è fondamentale** ricordare che:
Lo stato non va mai modificato direttamente.
React richiede sempre la creazione di **un nuovo array**, che rimpiazzerà quello precedente.

Difatti:

- quando aggiungiamo un elemento, costruiamo un nuovo array che contiene il nuovo valore e i valori precedenti;
- quando rimuoviamo un elemento, utilizziamo `.filter()` per ottenere un nuovo array privo dell'elemento selezionato.

La sintassi dello **spread operator** (`...prev`) consente di copiare rapidamente gli elementi precedenti nel nuovo array.

ⓘ **Info**

Metodi degli array più utilizzati in React

Nel codice proposto compaiono alcuni metodi ricorrenti quando si gestiscono liste dinamiche:

- `.includes()` → **verifica se un determinato valore è presente**
- `.filter()` → **restituisce un nuovo array escludendo specifici elementi**
- `.map()` → **trasforma ogni elemento dell'array in un corrispondente elemento JSX**

Questi tre metodi costituiscono la base del lavoro con gli array all'interno dei componenti React.

Esempio completo: `GroceryCart`

Nel seguente esempio viene mostrato come gestire lo stato di un carrello della spesa, aggiungendo e rimuovendo elementi tramite funzioni dedicate ed event handler.

Questo esempio è suddiviso in due file:

1. `ItemList.jsx` : il componente figlio

```
const ItemList = ({ items, onItemClick }) => {
  const handleClick = ({ target }) => {
    const item = target.value;
    onItemClick(item);
  };

  return (
    <div>
      {items.map((item, index) => (
        <button value={item} onClick={handleClick} key={index}>
          {item}
        </button>
      ))}
    </div>
  );
}
```

Spiegazione del codice:

1. **Firma del componente**
   `ItemList` riceve due *props* tramite destrutturazione:
   - `items` : array di valori da mostrare (es. `['apple','banana']`)

- `onItemClick` : funzione che il padre passa per essere notificato quando un elemento viene cliccato.

2. **Event handler** `handleClick`
   - Definito come `const handleClick = ({ target }) => { ... }` .
   - `target` è l'elemento DOM che ha generato l'evento (qui il `<button>`).
   - `target.value` contiene il valore del bottone (qui impostato con `value={item}`); lo assegna a `item` .
   - Poi chiama `onItemClick(item)` per comunicare al componente padre quale elemento è stato selezionato.
3. **Render con** `.map()`
   - `items.map((item, index) => (...))` trasforma ogni elemento dell'array in un `<button>` .
   - Ogni `<button>` ha: `value={item}` , `onClick={handleClick}` , `key={index}` e mostra `{item}` come testo.

> **Nota sulla** `key`
>
> - `key={index}` funziona, ma di norma è preferibile usare un identificatore stabile (es. `item.id`) quando disponibile.
> -Questo perché l'indice può causare problemi se la lista viene riordinata o elementi vengono inseriti/rimossi, di conseguenza l'indice è un parametro mutevole mentre gli id, essendo sequenziali, hanno come caratteristica l'immutabilità:
> se un elemento associato a un id viene inserito/rimosso il valore degli id degli elementi successivi e precedenti non cambia.

2. `GroceryCart.jsx` : componente padre

```
const ItemList = ({ items, onItemClick }) => {
  const handleClick = ({ target }) => {
    const item = target.value;
    onItemClick(item);
  };

  return (
    <div>
      {items.map((item, index) => (
        <button value={item} onClick={handleClick} key={index}>
```

```
          {item}
        </button>
      ))}
    </div>
  );
}
```

Spiegazione del codice

- **Stato `cart`**
  - `const [cart, setCart] = useState([]);` inizializza lo stato del carrello come array vuoto.
  - `cart` contiene gli elementi attualmente aggiunti; `setCart` aggiorna questo stato.
- **Aggiungere un elemento (`addItem`)**
  - `addItem(item)` viene passato ai figli (`ItemList`) come `onItemClick` .
  - All'interno chiama `setCart(prev => [item, ...prev])` .
    - Qui si usa la forma con *callback* (`prev => ...`) perché il nuovo stato dipende dal valore precedente: garantisce coerenza anche con aggiornamenti asincroni.
    - Crea un **nuovo array** (rispetta l'immutabilità), inserendo il nuovo `item` in testa all'array.
- **Rimuovere un elemento (`removeItem`)**
  - `removeItem(targetIndex)` riceve l'indice da rimuovere.
  - `setCart(prev => prev.filter((item, index) => index !== targetIndex))` crea un nuovo array escludendo l'elemento con quell'indice.
- **Render della lista del carrello**
  - `cart.map((item, index) => <li ... onClick={() => removeItem(index)} key={index}>...)` mostra ogni elemento come `<li>` cliccabile per rimuoverlo.
  - Anche qui: meglio usare chiavi stabili invece di `index` quando possibile.
- **Comunicazione figlio → padre**
  - Il flusso è: clic sul bottone in `ItemList` → `handleClick` legge `target.value` → chiama `onItemClick(item)` → `addItem` (nel padre) aggiorna lo stato del carrello.
  - Questo è l'esempio classico di *lifting state up;*

- comunicazione dall'alto verso il basso (props) e dal basso verso l'alto (chiamata di funzione passata come prop).

Deep

Osservazioni didattiche

- Lo stato `cart` tiene traccia degli elementi attualmente nel carrello.
- La funzione `addItem` costruisce un nuovo array utilizzando lo spread operator, mantenendo immutabile il precedente.
- La funzione `removeItem` utilizza `.filter()` per rimuovere selettivamente un elemento tramite il suo indice.
- La renderizzazione dinamica della lista avviene tramite `.map()`, generando un `<li>` per ogni elemento del carrello.
- La prop `key` consente a React di identificare stabilmente ogni elemento della lista, garantendo aggiornamenti efficienti nel Virtual DOM.
- Il componente figlio `ItemList` riceve sia l'array degli elementi da mostrare sia la funzione `addItem`, che può richiamare al click di un pulsante.

Lavorare con oggetti negli stati in React

Quando in un componente dobbiamo gestire più valori correlati(ad esempio più liste), è molto comodo raggrupparli all'interno di un oggetto nello stato.
Questo approccio permette di organizzare meglio i dati e rende più semplice aggiornare i valori, ad esempio in un form con diversi campi.

Esempio di utilizzo

```
import React, { useState } from 'react';

export default const Login = () => {
    const [formState, setFormState] = useState({});

    const handleChange = ({ target }) => {
        const { name, value } = target; // destrutturazione di name
e value dall'input; in sostanza i valori della coppia chiave-valore
corrispondo ai valori inseriti nel campo di input

        // Aggiorniamo lo stato usando una callback function
```

```
        // Lo spread operator copia tutti i valori già presenti
nello stato
        // e sovrascrive solo il campo modificato
        // In questo modo si evita di perdere i valori già aggiunti
sovrascrivendoli con la nuova coppia di valori appena aggiunti
        setFormState((prev) => ({
            ...prev,
            [name]: value
        }));
    };

    return (
        <form>
            <input
                value={formState.firstName || ''}
                onChange={handleChange}
                name="firstName"
                type="text"
                placeholder="First Name"
            />
            <input
                value={formState.password || ''}
                onChange={handleChange}
                name="password"
                type="password"
                placeholder="Password"
            />
        </form>
    );
};
```

In questo codice possiamo notare alcune cose:

1. **Callback function nello state setter**
   - Utilizziamo `setFormState(prev => ...)` per aggiornare lo stato in base al valore corrente. In questo modo ci assicuriamo di non sovrascrivere accidentalmente altri valori già presenti nello stato.
2. **Lo spread operator**
   - La sintassi `{ ...prev, [name]: value }` serve a copiare tutti i valori già presenti nello stato (`...prev`) e a sovrascrivere solo la chiave corrispondente all'input modificato.
3. **Gestore unico per più input**

- Grazie all'attributo `name` degli input, possiamo usare una sola funzione `handleChange` per aggiornare qualsiasi campo del form, senza scrivere gestori separati per ciascun input.

4. **Nome di proprietà calcolato**
   - Le parentesi quadre attorno a `[name]` indicano a JavaScript di usare il valore della variabile `name` come chiave dell'oggetto. Questo permette di aggiornare dinamicamente la proprietà corretta in base all'input che ha generato l'evento.
   - Per maggiori informazioni
5. **Stato immutabile**
   - Quando impostiamo un nuovo stato, non modifichiamo l'oggetto precedente.
   - Lo spread operator ci permette di creare un nuovo oggetto che include i valori precedenti e sovrascrive solo quelli aggiornati.
6. **Valori iniziali degli input**
   - Per evitare errori quando lo stato è ancora vuoto, possiamo usare `formState.firstName || ''` come valore degli input.
   - In questo modo, se lo stato non contiene ancora quella proprietà, il campo rimane vuoto invece di generare un errore.

Esempio pratico: `EditProfile`

```
import React, { useState } from 'react';

const EditProfile = () => {
    // 1. Dichiaro e inizializzo lo stato come oggetto vuoto
    const [profile, setProfile] = useState({});

    // 2. Event handler per gestire le modifiche agli input
    const handleChange = ({ target }) => {
        const { name, value } = target; // destrutturazione di name
e value dall'input

        // Aggiorno lo stato senza perdere i valori già presenti
        setProfile((prevProfile) => ({
            ...prevProfile,
            [name]: value
        }));
    };
```

```
    // 3. Event handler per gestire il submit del form
    const handleSubmit = (event) => {
        event.preventDefault(); // evita il reload della pagina
        alert(JSON.stringify(profile, '', 2)); // mostra i dati
inseriti
    };

    // 4. Ritorno JSX
    return (
        <form className="form-group" onSubmit={handleSubmit}>
            <input
                className="form-control"
                value={profile.firstName || ''}
                name="firstName"
                type="text"
                placeholder="First Name"
                onChange={handleChange}
            />
            <br />
            <input
                className="form-control"
                value={profile.lastName || ''}
                name="lastName"
                type="text"
                placeholder="Last Name"
                onChange={handleChange}
            />
            <br />
            <input
                className="form-control"
                value={profile.bday || ''}
                name="bday"
                type="date"
                onChange={handleChange}
            />
            <br />
            <input
                className="form-control"
                value={profile.password || ''}
                name="password"
                type="password"
                placeholder="Password"
                onChange={handleChange}
            />
```

```
            <br />
            <button type="submit">Submit</button>
        </form>
    );
};
export default EditProfile;
```

Spiegazione passo passo

1. **Dichiarazione dello stato(** `const [profile, setProfile] = useState({});` **)**
   - `profile` → **contiene i dati del form come oggetto.**
   - `setProfile` → **è la funzione che aggiorna lo stato.**
   - **Inizializziamo come oggetto vuoto perché i campi del form non hanno valori iniziali.**
2. **Gestione delle modifiche agli input (** `const handleChange = ({target})=> {...}` **)**
   - **La funzione riceve l'evento generato dall'input.**
   - `target` **rappresenta l'elemento** `<input>` **che ha generato l'evento.**
   - **Con la destrutturazione otteniamo** `name` **e** `value`**.**

📋 Destrutturazione `name` / `value`

Quando gestiamo eventi su `<input>` in React, possiamo ottenere facilmente `name` e `value` dall'oggetto `target` dell'evento in due modi:

1. **Metodo Classico:**

```
const name = target.name;
const value = target.value;
```

- Funziona perfettamente.
- Richiede due righe di codice.
- Prende manualmente le proprietà dall'oggetto `target`.

2. Metodo compatto con destrutturazione:

```
const { name, value } = target;
```

Questo metodo fa la stessa del primo metodo, le differenze e i vantaggi sono:

- Più conciso e leggibile.
- Utile quando vuoi estrarre più proprietà contemporaneamente.

> **Nota: la destrutturazione è solo una scorciatoia sintattica: sotto il cofano crea le stesse variabili `name` e `value` pronte per essere usate nel tuo codice.**

- Aggiorniamo lo stato con `setProfile((prevProfile) => ({...}))`:
  - Lo **spread operator** `...prevProfile` copia tutti i valori già presenti nello stato.
    - In questo modo, come per `GroceryCart`, si evita di sovrascrivere i valori già presenti con quello appena aggiunto

```
- `[name]: value` aggiorna solo la proprietà corrispondente
all'input modificato.
```

    - Le parentesi quadre intorno a `name` permettono di usare il valore della variabile come chiave dell'oggetto (**computed property names**).

3. **Gestione del submit (** `const handleSubmit = (event) =>{...}` **)**
   - `event`: è il parametro in input che corrisponde all'oggetto evento generato dal `<form></form>`
   - `event.preventDefault()` impedisce il comportamento predefinito del form (ricaricamento della pagina).
   - `alert(JSON.stringify(profile, '', 2))` mostra i dati del form in una finestra di alert, formattati con indentazione per una migliore leggibilità.
4. **Valori degli input**
   - Ogni input ha `value={profile.nomeCampo || ''}`, come valore di default:
     - in sostanza serve per evitare errori quando lo stato è ancora vuoto.
     - prende in input il valore della chiave `fisrtName` nell'oggetto `profile`
     - Se `profile.firstName` è `undefinded`, `null` o qualsiasi valore "falsy"; allora restituisce una stringa vuota (`''`)
     - Quindi finché l'utente non ha inserito nulla, il campo sarà vuoto.

    - Appena scrive qualcosa, `profile.firstName` viene aggiornato e l'input mostra quel valore.
  - L'attributo `name` identifica il campo corrispondente nello stato.
  - L'evento `onChange={handleChange}` collega ogni input al gestore unico, così possiamo aggiornare dinamicamente lo stato.
5. **Submit del form**
  - Il bottone `<button type="submit">Submit</button>` invia il form.
  - Grazie a `onSubmit={handleSubmit}` il submit è gestito da React, evitando ricariche indesiderate e permettendo di leggere i dati dallo stato.

Hooks separati per stati separati

A volte può essere utile memorizzare dati correlati in un'unica struttura (array o oggetto). Tuttavia, quando alcune parti cambiano indipendentemente dalle altre, è spesso preferibile creare stati separati.

Gestire dati dinamici è più semplice quando il modello di stato rimane il più lineare possibile.

Esempio iniziale: uno stato oggetto unico

Ad esempio:
Se abbiamo un singolo oggetto che mantiene uno stato per una materia che si sta studiando a scuola, esso potrebbe assomigliare a qualcosa di simile:

```
// dichiariamo un componente
const Subject = () => {

    // al suo interno dichiariamo e inizializziamo uno stato con un
oggetto non vuoto
    const [state, setState] = useState({
        currentGrade: 'B',
        classmates: ['Hasan', 'Sam', 'Emma'],
        classDetails: { topic: 'Math', teacher: 'Ms. Barry', room:
201 },
        exams: [
            { unit: 1, score: 91 },
            { unit: 2, score: 88 }
        ]
    });
```

```
};
```

Spiegazione passo-passo:

1. **Dichiarazione del componente** — `const Subject = () => { ... }` definisce il componente funzionale.
2. **Uso di `useState` con un oggetto** — `const [state, setState] = useState({...})` crea un unico stato (`state`) che raggruppa più proprietà (voto attuale, compagni, dettagli, esami).
3. **Struttura dei dati** — nell'oggetto sono presenti:
   - `currentGrade` : valore singolo (stringa);
   - `classmates` : array di stringhe;
   - `classDetails` : oggetto annidato con più campi;
   - `exams` : array di oggetti, ciascuno rappresenta un esame.

✓ **Vantaggio principale**

- **Compattezza:**
  - tutti i dati relativi alla materia sono in un solo oggetto, utile se si vuole passare tutto come unica unità ad altri componenti.

× **Limite pratico**

quando si aggiornano parti profonde (es. un singolo `score` dentro `exams`) si deve ricopiare l'intero oggetto o le parti modificate, aumentando la complessità del codice di aggiornamento.

Problema pratico: aggiornare un esame

Questo potrebbe funzionare, *ma **attenzione***:

- questo codice sebbene risulti molto più compatto mano a mano che lo si va ad ampliare può presentare diverse problematiche
- Se ci riflettessimo un po' di più sopra la prima domanda che sorge spontanea è: "*come dovrei fare a copiare tutti questi valori quando ho necessità di aggiornare qualcosa in questo oggetto di stato cosi grande?*"
  Per esempio:

Per aggiornare il voto di un esame, avremo bisogno di un event handler che fa qualcosa di simile:

```
// richiamo la funzione state setter ('setState') passando il valore precedente
setState((prev) => ({

    // espando tutti i valori dell'oggetto precedente
    ...prev,

    // itero sui valori della chiave 'exams' tramite .map()
    exams: prev.exams.map((exam) => {

        // check: se l'unit dell'esame corrente corrisponde all'esame aggiornato
        if (exam.unit === updatedExam.unit) {

            // restituisco i valori precedenti dell'esame espandendoli
            return {
                ...exam,

                // aggiorno il valore dello score
                score: updatedExam.score
            };

        } else {

            // altrimenti restituisco semplicemente l'esame così com'è
            return exam;
        }
    }),
}));
```

Spiegazione passo passo:

1. **Uso della callback nello state setter** — `setState((prev) => {...})` utilizza lo stato precedente (`prev`) per calcolare il nuovo stato in modo sicuro (evita race condition quando più aggiornamenti sono in coda).
2. **Copia dello stato precedente** — `...prev` crea un nuovo oggetto che include tutte le proprietà esistenti, preservando i campi non modificati.

3. **Aggiornamento mirato di** `exams` — si assegna alla proprietà `exams` il risultato di `prev.exams.map(...)`, cioè una nuova array derivata dalla precedente.
4. **Iterazione e condizione** — per ogni `exam` in `prev.exams` si verifica se `exam.unit === updatedExam.unit`:
   - se vero: si ritorna un nuovo oggetto `exam` copiando le proprietà esistenti (`...exam`) e sovrascrivendo `score` con il nuovo valore;
   - se falso: si ritorna l'oggetto `exam` originale (immutabilità preservata).
5. **Risultato** — il nuovo stato è un oggetto con tutte le proprietà precedenti, ma con `exams` sostituito da una nuova array che contiene l'esame aggiornato.

> **Perché è complesso**
>
> bisogna gestire copie (spread), map, e condizioni: più livelli di annidamento significano più possibilità di errori/bug e codice più verboso.

Anche in questo codice nulla di sbagliato in realtà, tuttavia un codice così grande è complesso può portare ad una serie di errori e causare bug.

Di conseguenza una buona prassi in React è quella di:

- **creare più variabili di stato in base ai valori che tendono a cambiare insieme.**

Quindi riscriviamo l'esempio precedente in modo più semplice:

```
const Subject= () => {
    const [currentGrade, setGrade] = useState('B');
    const [classmates, setClassmates] = useState(['Hasan', 'Sam',
'Emma']);
    const [classDetails, setClassDetails] = useState({
        topic: 'Math',
        teacher: 'Ms. Barry',
        room: 201
    });
    const [exams, setExams] = useState([
        { unit: 1, score: 91 },
        { unit: 2, score: 88 }
    ]);
}
```

Spiegazione passo-passo:

1. **Separazione logica** — ogni porzione di dati ha ora il proprio `useState`: voto, compagni, dettagli della classe ed esami.
2. **Semplicità negli aggiornamenti** — aggiornare `exams` richiede ora solo `setExams(...)`, senza preoccuparsi di copiare l'intero oggetto `state`.
3. **Funzioni setter dedicate** — `setGrade`, `setClassmates`, `setClassDetails`, `setExams` permettono aggiornamenti isolati e più leggibili.
4. **Testabilità e debug** — errori legati a una specifica parte dello stato sono più facili da localizzare; i test possono mirare a funzioni/setter singoli.
5. **Trade-off** — più hook `useState` possono aumentare il numero di variabili da gestire nel componente, ma generalmente migliorano la manutenibilità se i dati cambiano indipendentemente.

Quindi maneggiare dati in modo dinamico con variabili di stato separate ha numerosi vantaggi:

1. rende il codice più leggibile
2. più facile da scrivere
3. facilita il testing e il debugging
4. maggiore riusabilità del codice tra diversi componenti

Spesso ci troviamo a raggruppare e organizzare i dati in raccolte da trasferire tra componenti, per poi separare tali dati all'interno dei componenti in cui le diverse parti cambiano separatamente.

La cosa fantastica degli Hooks è proprio questa:
abbiamo la libertà di organizzare i nostri dati in maniera tale da avere senso per noi.

Esempio pratico : `Musical`

```
const Musical = () => {
    const [state, setState] = useState({
        title: "Best Musical Ever",
        actors: ["George Wilson", "Tim Hughes", "Larry Clements"],
        locations: {
            Chicago: {
                dates: ["1/1", "2/2"],
                address: "chicago theater"
            },
```

```
            SanFrancisco: {
                dates: ["5/2"],
                address: "sf theater"
            }
        }
    });
};
```

Spiegazione passo-passo:

1. **Stato unico e annidato** — titolo, attori e locations (oggetti annidati per città) sono raggruppati in un solo oggetto di stato.
2. **Aggiornamenti potenzialmente complessi** — per modificare `actors` o una singola `location` occorre ricreare parti dell'oggetto, con spread su `state` e/o su `locations`.
3. **Comodità** — utile quando si desidera passare lo stato completo come singolo prop ad un figlio o serializzarlo insieme.
4. **Svantaggio** — al crescere della struttura la gestione e gli aggiornamenti diventano verbosi e più inclini a errori.

Anche in questo caso se dovessimo aggiornare insieme `actors` e `locations` diventerebbe complicato e lungo da fare senza, tra l'altro, avere garanzie che questi aggiornamenti non possano causare big.
Di conseguenza proviamo a rifare un refactoring di questo codice separando `actors` e `locations` in due stati separati

```
const MusicalRefactored = () => {
    const [title, setTitle] = useState("Best Musical Ever");
    const [actors, setActors] = useState([
        "George Wilson",
        "Tim Hughes",
        "Larry Clements"
    ]);
    const [locations, setLocations] = useState({
        Chicago: {
            dates: ["1/1", "2/2"],
            address: "chicago theater"
        },
        SanFrancisco: {
            dates: ["5/2"],
            address: "sf theater"
        }
```

```
    });
};
```

Spiegazione passo-passo:

1. **Separazione dei concetti** — `title`, `actors`, `locations` sono ora gestiti da hook separati.
2. **Aggiornamenti mirati** — per aggiungere/rimuovere un `actor` basta chiamare `setActors(...)`; per aggiornare una città si usa `setLocations(...)`, isolando i cambiamenti.
3. **Migliore leggibilità** — i setter sono più piccoli e il codice che li utilizza è più semplice da leggere e mantenere.
4. **Quando preferire questo approccio** — **quando le parti di stato cambiano in modo indipendente o quando si vogliono riusare singole parti in sotto-componenti.**

Ora difatti possiamo aggiornare tranquillamente questi due stati senza dover scrivere codice verboso e difficile da debuggare.

In sintesi:

1. **Usa uno stato oggetto** quando: vuoi raggruppare dati che vengono quasi sempre modificati insieme o vuoi passarli come unica entità ai figli.
2. **Separare gli stati** quando: le porzioni di dati cambiano indipendentemente, vuoi semplificare gli aggiornamenti e facilitare testing/debugging.
3. **Regola pratica**: preferisci semplicità e chiarezza del codice. Se un singolo aggiornamento diventa verboso e complesso, valuta il refactoring in stati separati.